ITERATING PHP ITERATORS

An eBook by Cal Evans

learnonethingbooks.com

Iterating PHP Iterators

Cal Evans

This book is for sale at http://leanpub.com/iteratingphpiterators

This version was published on 2015-04-10

* * * * *

This is a Leanpub book. Leanpub empowers authors and publishers with the Lean Publishing process. Lean Publishing is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

* * * * *

© 2013 - 2015 Cal Evans

Table of Contents

Dedication

Chapter 1 – The What and Why of Iterators
- What Are Iterators
- Iterators vs. Iterable Interface
- Why Use Iterators
- Why Not Just Use Arrays
- Two Kinds Of Iterators
- Source Code
- Summary

Chapter 2 - The Iterator Interface
- Why Not Just Use The Built-in Iterators
- Building Your Own Iterator
- The Countable Interface
- Summary

Chapter 3 - How Iterators Work
- How An `Iterator` Operates
- Summary

Chapter 4 - Outer Iterators
- Pre-made Outer Iterators
- Creating An `OuterIterator`
- Summary

Chapter 5 - The ArrayIterator
- `ArrayObject`
- `ArrayIterator`
- Why Use An Iterator For An Array?
- Offsets
- Flags
- Summary

Chapter 6 - …Of Inputs And Outputs
- Modifying Output
- Restricting Inputs
- Summary

Chapter 7 - The SeekableIterator Interface
- Summary
- Bonus

Chapter 8 – Recursing The Iterator

New Methods
`RecursiveIteratorIterator`
The Most Common Use Case
Summary

Chapter 9 - Iterating The File System
`DirectoryIterator`
`FileSystemIterator`
`RecursiveDirectoryIterator`
Summary

Chapter 10 - The Caching Iterator
`CachingIterator` And “Look Ahead”
Flags
Summary

Chapter 11 - Generators
What Are Generators?
A More Complete Example
Summary

Appendix A - A Bit Math Primer
Starting Small
Bigger And Better
Summary

NomadPHP

Dedication

To the PHP community. You’ve given me so much.

Chapter 1 – The What and Why of Iterators

Welcome to the wonderful world of Iterators!

PHP has long had a deep toolbox for dealing with data stored in arrays. Because of this, many developers never look beyond the native functions for processing lists of items that have to be iterated over. In this book we will look at the oft-forgotten tool, iterators. This does not mean that you as a developer now have to switch to processing all of your lists with iterators. They don't replace native array functions, or anything else for that matter; they simply add to tools you already have available to you.

What Are Iterators

If you have ever used a `foreach` loop in PHP, you have used an iterator. In a `foreach` loop, the first parameter passed is whatever you are iterating over, and therefore an `Iterable`. Most developers never use more than arrays in a `foreach` and that is fine. However, there are times when it is useful to be able to iterate over your own classes. In many cases, the task you want to accomplish is already encapsulated in a pre-built class in PHP's Standard PHP Library. The SPL has pre-built special case iterators that, because they are part of the core language, run faster than any code you could write in "userland".

> Userland is PHP code that you write and the engine interprets.

Iterators vs. Iterable Interface

In the second chapter of this book we will actually build a class that implements the Iterator interface. `Iterator` is an interface built into PHP that defines the methods that have to be present for an object to be used as an `Iterator`. As we will discover, there are only five methods needed for a class to implement `Iterator` interface. In the basic example we give, all of them are simple to write and understand. While this section is only a small part of the overall book, it is an important part because it lays the foundation for your understanding of what iterators are and the power they give you.

After you understand what an `Iterator` is from the code side of things, we look at the iterators that are built into PHP's SPL. The rest of the book is dedicated to these classes. Iterators are powerful, ready-made classes that, once you understand them, will change the way your write code. Together, these two new tools will help you build better, faster applications in less time.

Why Use Iterators

`Iterators` give you the ability to have objects – whether of your own design or part of the SPL – that can replace arrays in any situation where you can use an iterable. As we will show in later chapters, this gives you fine-grained control over the loop. Additionally, you will find that you can insert your own code in the process.

In some cases, the DirectoryIterator being the most common example, an iterator replaces code that you would normally write. The `DirectoryIterator` gives us an easy and consistent way to walk a directory tree of a file system. Most developers at one time or another have written code to do this very task. Sometimes it's good, sometimes it's bad, and sometimes all you can say is it gets the job done. By implementing the `DirectoryIterator` you know exactly how the code will run. More importantly, the next developer who comes along behind you knows as well.

Finally, many developers avoid using iterators because they are afraid that it's too complex. The truth, however, is that once you learn how to use them, you will find yourself creating projects faster. The problem with iterators is not that they are overly complex, it is that in many cases, they have been poorly explained. Hopefully, this book will solve that problem.

Why Not Just Use Arrays

When dealing with iterators, invariably the question comes up, "why not just use arrays?" It's a valid question. In some cases an array is an acceptable solution. However, if all you ever iterate over is an array then you end up writing a lot of code inside your `foreach` to process the contents of the array. For instance, if you are iterating over a list of database rows, you have to write the code to fetch the row from the database each time. It would be more efficient to write that code once and store it with the array. This is called encapsulation and is a basic tenant of Object Oriented Programming (OOP). However, in PHP, unless you understand Iterators, you have to make a tradeoff. You can either have the ease of use of an iterable array, or the reuse of an un-iteratable object.

The other important difference between an array and an iterable object is that with the object you, the developer, get to define what data is iterated over, and how. If you drop an array into a `foreach`, it will start at the top element, drop through to the bottom element, and execute your code for each element in between. In an iterable object, since you have control over your object, you can iterate over a property that is an array, over individual properties of the object, or your object can interact with other objects to fetch data on an as-needed basis and iterate through that data. However you want your object to behave, you have control over it.

Thankfully in PHP, you can have both — and more. That is what we will be looking at in this book: what Iterators are and, more importantly, how you can begin to use them in your code.

Two Kinds Of Iterators

PHP's SPL actually has two kinds of Iterators, those you use, and those you 'implement'. The latter is what we will focus on in this book. PHP's excellent documentation is more than sufficient.

Source Code

To help demonstrate the concepts that we will be discussing, I have created a sample code repository on Github titled Iterators Most of the code in the samples won't actually appear

in this book. It's easier for you to simply fork the repo and play with it than have to cut-and-paste and create your own version from scratch.

If you clone the repo, fork the repo, or just download the zip file, know that before it will work, you will have to do a `composer install` to pull in all the dependencies.

> It should be noted that the sample code written for this book is just that, **sample code**. This code is not intended for you to cut-and-paste into production projects. This code does no input validation, no export filtering and no error handling. It is meant solely to demonstrate the points being discussed. **Do not use this code as-is in any project, ever.**

Prerequisites

- PHP 5.6+
- A solid foundation in Object Oriented Programming in PHP
- composer
- git *
- A good text editor *

> - These aren't required; they will however make working with the code in this book easier.

I've made every effort to make sure the code provided is clean, easy to understand and bug-free. However, if you find a bug, or a better way to demonstrate a given concept, please feel free to submit a pull request.

Summary

You skipped everything in this chapter and just jumped to the summary didn't you? That's ok in this chapter but it won't work in most of the chapters. So here is what you missed.

In this opening chapter we discussed:

- Iterators allow you to use your own objects in a `foreach` loop.
- Iterators built into the SPL help us standardize common tasks so that they are easier to understand and will run faster.
- Iterators implement the `Iterator` interface and we as developers can write classes that implement it as well.
- Iterators give us the best of both worlds, the convenience of an array with the encapsulation of an object.

Chapter 2 - The Iterator Interface

We now know *why* we should be interested in iterators. Let's dig into the meat of things and talk about how to build one ourselves.

Before we dive in, remember, if you want to play with the code you will need to clone the github repo. We will examine relevant snippets of the overall code but this book will not contain the entire codebase.

Why Not Just Use The Built-in Iterators

In this chapter we look first at building our own iterator. The SPL has 24 built-in iterators, so the question must be asked, why would you want to write one yourself? The simple answer is that no matter how well-meaning and smart the PHP core developers are, they cannot plan for every situation. Knowing this, they have given us the ability to make our own classes iterable. The `Iterator` interface is the way we do that. If we understand the `Iterator` interface and how to implement it then we can handle any situation over which we need to loop.

Let's create a very simple class to explain.

```
class Car
{
        protected $cylinders     = 4;
        protected $color          = 'black';
        protected $doors          = 2;
        protected $convertible   = true;
        protected $transmission = 'manual';
        protected $seats          = 2;
}
```

In this example we've defined a simple class describing the basic attributes of my personal car. As it is right now, I can use the class to store that information but if I want to print it all out, I have to manually write the code that prints each property. It would be nice if I could simply drop it into a `foreach` and spin through the properties, outputting each one in sequence. That is what `Iterable` gives us the ability to do… kind of.

```
class Car implements Iterator
{
        protected $cylinders     = 4;
        protected $color          = 'black';
        protected $doors          = 2;
        protected $convertible   = true;
        protected $transmission = 'manual';
        protected $seats          = 2;

        public function key()
        {

        }
```

```
        public function next()
        {

        }

        public function rewind()
        {

        }

        public function valid()
        {

        }

        public function current()
        {

        }
}
```

Now we are on our way. Note we have added stubs for the five methods necessary to implement the `Iterator` interface. If we were to flesh out these methods properly, we could drop the resulting object in a `foreach` and iterate over the properties. The devil, as they say, is in the details.

One hard part in a class implementing `Iterator` is that the class itself has to:

- Maintain its own internal pointer to the current property and be able to return the pointer to the current key, and the current value.
- Know what the next property is to be iterated over
- Know when it's run past the end of the property list
- Know how to reset the pointer to the top of the list

In other words, your class now has to know how to operate as an array. You could, through PHP's Reflection API, devise very clever code that would cause the object to inspect itself, build a list of the properties, and then use that list to maintain state. However, at that point, you have a class wrapping an array that is dynamically built each time it is instantiated, regardless of what else needs to be done. In a dynamic scripting language, this is expensive.

A common solution is to design your class so that your properties are stored in an array.

```
class Car implements Iterator
{
    protected $data = [
        'cylinders'     => 4,
        'color'          => 'black',
        'doors'          => 2,
        'convertible'   => true,
        'transmission' => 'manual',
        'seats'          => 2,
    ];
```

```
    // ...
}
```

Many developers already do this as the values for the properties are coming from external sources like a database. It also makes using the magic `__get()` and `__set()` methods much easier. When all your iterable properties are stored in an array, making your class iterable is much easier.

Building Your Own Iterator

So now that we know how easy it is to make an existing class iterable, let's look at the example code that you should have already downloaded.

In this example, we will be bringing in data from the Github API, storing it, iterating over it, and extending it on an as-needed basis. Our sample code pulls in the latest commits to PHP itself and lists out the contributors and where they are located, if known.

To do this, we have to make calls to two separate API endpoints. First we call for the list of recent commits to the language. Then, to get the location data, we make a call to the user endpoint. In our relatively simple example, we never make more than five to ten calls per run, well under the limit. However, in larger examples, you might be making more.

There are two ways you can approach this. First, after you populate the array of contributors, you can make all the user calls and populate the location data. However, we've all written classes that are instantiated and for whatever reason - an exception thrown during the main script, a branching call, any number of reasons - the object is never used. If you are populating all of the data, then you are making a lot of unnecessary calls.

The second way is to make the call to the commits endpoint and build out our base list, but not make the individual calls to the user endpoint until we actually need them. This technique is known as Lazy Loading. The other advantage to this way is speed. Whether you are fetching your data from an external source like an API or a local database server, calls like this are expensive in execution time. The fewer of them we can make, the faster our scripts will operate. Therefore, that is the approach we take in our sample code.

```
class IteratorDemo implements Iterator, Countable
{
        protected $commiters = [];
        protected $pointer    = -1;

    // ...
}
```

First, notice that we are implementing both the `Iterator` interface as well as the `Countable`. This is not strictly necessary for our example, it's just a nice touch. By implementing `Countable` we guarantee that the user can use it in any situation that requires the `Countable` interface. The side effect is that because we have to implement the `count()` method, the user can always find out how large our list of properties is.

The only two properties of this class are the list of `committers` and the `pointer`. If you have used arrays before in PHP then you are aware that it maintains a pointer for each array. Technically, we could use the built-in PHP functions that give us access to that

pointer, and not keep our own. However, the point of building an iterable is to take control. So we will take responsibility for the pointer ourselves.

We won't spend too much time on the methods that deal with the API as that's not why we are here. As you can see, if you have the source code open, the constructor populates the array of committers with a call to `$this->populate()`. Then to make sure everything is reset *and* to fetch the location data on the first record, it calls `$this->rewind()`.

`rewind()` is one of the five methods you have to create to implement the `Iterator` interface. Let's list them here and describe them. In no particular order, they are:

- `rewind()` - This sets your pointer to `0`, effectively resetting your array to the top for the next time it will be used.
- `key()` - This returns the current key. In our case, since the `$committers` array is indexed and not associative, it returns the value of `$this->pointer`. If `$committers` were an associative `array, you would return the current key of the array. When you specify` foreach ($this as $key ⇒ $value) key() `is the method used to retrieve the` $key`.
- `current()` - This method will return the value of the current array element. When you specify `foreach ($this as $key => $value)` `current()` is the method used to retrieve the `$value`.
- `next()` - This moves the pointer. In our case, because ours is an indexed array, this means we increment the pointer. If you were using an associative array, you would move the pointer to the next key value.
- `valid()` - Valid will always return `true` or `false`, based on whether the pointer is outside the range of valid keys or not. After calling `next()` to increment the pointer, PHP calls `valid()` to know whether it is at the end of the list or not. When `valid()` returns `false`, the `foreach` exits its loop. It's that simple.

In each of these methods, you can also add your own code, as we will see a little further along.

Part of the power of creating your own iterable class is being able to control behavior. For instance, in most cases, `next()` on an indexed array will increment the pointer by `1`. There is no rule that says you have to though. You could increment it by `2`, `5`, or whatever value you wish. You could even parameterize it so that at runtime, you decide how many steps `next()` takes.

You could also define custom rules for `valid()`. You could make `valid() return its normal value except when it's called exactly at midnight. You could make` valid()` return false if you lost access to an external resource like your database or API. (Not a great idea, but it could be done) There are any number of ways you can now exert control over the iteration of your object.

Be very careful with `valid()`. If for some reason you code a class that always returns `true` from `valid()`, or returns `true` when the pointer has actually gone beyond the list of parameters, your `foreach` loop will still continue to run, even though it is not giving you back any data. It will continue to run until you manually exit the code or the PHP script timer kills it.

Now that you know the pieces, you can look at the sample code and see how they are implemented. There are two of the methods that contain code outside of that necessary for the `Iterator` interface to function. Let's look at those in depth.

The first one we run into is `rewind()` as it is called from the constructor.

```
public function rewind()
{
        $this->pointer = 0;

        if (count($this->committers) > 0) {
                $this->committers[0]['location'] = $this->fetchLocation(0);
        }
}
```

After the constructor method calls `populate()`, it calls the rewind method above. This does two things. First, it sets the pointer to `0`, the part that is necessary for the `Iterator` interface. Then, if we have committers - safety in case there was nothing returned - we call `fetchLocation()` on the current committer, which is at the top of our list.

`fetchLocation()` is designed in such a way that it won't make calls unnecessarily. So once the call has been made, it will simply return the current value. This saves us hits on the github API as well as time.

We make the call here because if we didn't, then the first element would never have a location attached to it. We could make this call at the end of `populate()` passing in a value of `0` to achieve the same results. However, it seemed more natural to make the call here in `rewind()`.

The other place we have added code beyond that necessary to satisfy the `Iterator` interface is in the `next()` method.

```
public function next()
{
        $pointer = ++$this->pointer;

        if ($this->valid()) {
                $this->committers[$pointer]['location'] = $this->fetchLocation($pointer);
        }
}
```

Similar to what we do in `rewind()`, we make a call to `fetchLocation()` to populate the location of the committer in `next()` show above. The call in `rewind()` populates it for the first committer, after that, this is the natural place to make the call. In the code above, the first line increments our internal pointer. Then, before we make the call to `fetchLocation()` using the new pointer, we check to make sure the new pointer is valid. If we did not do this, due to the way that PHP operates, the `foreach` would never end. We would be setting `$this->committers[$this->pointer]['location']`. If `$this->pointer` were beyond the end of our array, PHP would just add a new element to the committers array. This would ensure an endless loop until the script timer ran out or your computer's memory filled up.

The Countable Interface

The other interface we are implementing in this example is the `Countable` interface. This allows you to pass your object to methods like `count()` and have it return a value. In our case, we are returning the length of the committers array. However, since you have control, you can return any value you choose.

Implementing `Countable` requires you to define a single method `count()`.

```
public function count()
{
          return count($this->committers);
}
```

If you have downloaded the source code, you will find a command to exercise this class we've created.

```
$ ./console iteratorInterface
```

This should output a list of recent committers to the PHP language and where they are from (if they have put that information in their profile).

The code to exercise our object is straightforward.

```
$iteratableClass = new Iterators\Classes\IteratorDemo();

$output->writeln(
     'Looks like we've got ' . count($iteratableClass) . ' recent committers.'
);

foreach ($iteratableClass as $committer) {
        $output->writeln(
        'Commiter : ' . $committer['name'] . ' from ' . $committer['location']
    );
}

$output->writeln('Done');
```

Notice that the first thing we do is pass the object into the `count()` method, thus demonstrating the `Countable` interface. Had our class not implemented `Countable` then PHP would not have thrown an error; it would only see the one object to count, therefore returning `1`.

Next is the `foreach` that exercises the `Iterator` interface. Again, if our class doesn't implement `Iterator` PHP won't necessarily throw an error, but it won't know how to iterate over our object, so it will simply end the `foreach`.

Summary

In this chapter we have created our own iterable object and shown how to use it. This is a very simple example but it demonstrates how to incorporate your own code into the five required methods, as well as add additional methods to the class that can be used to make decisions and branch code.

- Iterators are easy to create. You only have to implement five simple methods.
- Implementing the `Iterator` interface allows your object to be used anywhere PHP expects an `iterator`, such as a `foreach` loop.

- You have control over how the iterating works inside of your class. You can change the rules to fit your specific needs.

Chapter 3 - How Iterators Work

Now that we have created our first iterator, let's take a look at exactly how things work. In our example, we created a class that implemented the `Iterator` interface. We then dropped that class into a `foreach` loop and spun through it. We saw from the output that everything worked correctly and that we were able to add our own code into it. The one thing we didn't explore is when, how, and in what order, all the methods are fired.

How An `Iterator` Operates

To understand the process, let's think about how the `FilterIterator` works internally. Normally, if you drop an iterable item into a `foreach` loop, the loop does this:

```
foreach ($myThing as $thisThing) {
    echo $thisThing . "\n";
}
```

1. Internally PHP first calls `rewind()` to reset the pointer of the iterator. In our case, since we have an [`OuterIterator`] (http://www.php.net/manual/en/class.outeriterator.php, that is called the `OuterIterator` and passed down to the main `Iterator`.
2. `accept()` is called on the first element in the inner `Iterator`. If `false` is returned, the inner `Iterator`'s `next()` is called to move the pointer and the outer iterator's `accept()` method is called again. This continues until the end of the inner iterator is reached, or `accept()` returns `true`.
3. Once `accept()` returns `true`, the `OuterIterator`'s `valid()` is called to ensure we are looking at a valid element.
4. When `valid()` returns `true`, the `OuterIterator`'s `current()` is called to return the value of the current element.
5. Then the code inside the `foreach` is executed.

We could have also written our `foreach` to return the current key as well.

```
foreach ($myThing as $key => $thisThing) {
    echo $thisThing . "\n";
}
```

This would have added a sixth step to our list above where `key()` would have been called.

So now that we know the theory, let's see it in action. Execute the sample we played with in the previous chapter but add the `-v` switch. This will be passed into our `IteratorDemo` object and each method will report when it is being called. This will show us the order in which they are being called so we can understand how things are working.

```
$ ./console iteratorInterface -v
```

This will now generate a lot more output. You will see lines beginning with `--`. These lines announce when a method is fired.

Let's look at a small sample of the output.

```
Committer : Julien Pauli from Paris—Next—Valid—Fetch Location—Valid—Current
Committer : Andrea Faulds from United Kingdom
```

- We see the output from the `foreach` loop announcing the contributor and location.
- Then we see that `next()` is fired to move the pointer.
- We see that `valid()` is called.
- Our custom code `fetchLocation()` is called to populate the location field if necessary.
- `valid()` is fired again.
- `current()` is fired to fetch the value of the current element in the iterator.
- Finally the next core contributor is output by the `foreach`.

Now in the above sequence, something is out of whack. In our verbose output, `valid()` is fired twice. This is actually correct. The first time `valid()` is fired, it is not part of the `Iterator` class firing it. Instead it is our own custom code in the `next()` method calling it.

```
public function next()
{
        if ($this->verbose) {
                echo "-- Next\n";
        }

        $pointer = ++$this->pointer;

        if ($this->valid()) {
                $this->committers[$pointer]['location'] = $this->fetchLocation($pointer);
        }
}
```

Notice that we actually make a call to `$this->valid()` in this method. If we didn't - if we didn't check to make sure that the pointer was actually pointing to an existing array element - then when we did hit the end of the array `$this->committers` this code would actually add a new element onto the end of the array each time it is called. This would create a never-ending loop.

```
$this->committers[$pointer]['location'] = $this->fetchLocation($pointer);
```

Once the end of the array is hit, and `$this->pointer` is incremented, it is now pointing to an element of the array that does not exist. As long as all we do is test this, we are fine. However, because of how PHP works, trying to assign a value to `$this->committers[$pointer]['location']` would actually create a new element in the array. So, we check to see if we have a valid pointer before we do this. It would have been simple in this case to check the value of `$this->pointer` without calling `valid()`. However, in a more complex scenario, there may be other rules in play to determine whether the pointer was valid or not. It is best to group them all in `valid()` and fire the method whenever the tests need to be run.

Summary

By now I hope you are beginning to see that iterators are not complex to create. We've created one and we've see exactly how it works. Take a look at the sample code, take it

apart, add your own code into the mix. Play with it until you understand not only how things work but why they work that way. This is the foundational knowledge that we build upon in coming chapters.

- An `Iterator` has five required methods.
- We can control what goes on inside an `Iterator` with our own code.
- It is stupidly easy to create an endless loop if you aren’t careful.

Chapter 4 - Outer Iterators

One special class of iterators in PHP are the “Outer Iterators.” These are iterators that wrap other iterators and perform some special function in addition to just iteration. In addition to the five main methods of an `Iterator`, the `OuterIterator` interface specifies that `getInnerIterator()` be present.

Pre-made Outer Iterators

The SPL contains 13 implementations of the `OuterIterator` interface. Some of these implementations are interfaces, such at the `FilterIterator` and others are concrete classes, such as the `AppendIterator`.

- `AppendIterator`
- `CachingIterator`
- `CallbackFilterIterator`
- `FilterIterator`
- `InfiniteIterator`
- `IteratorIterator`
- `LimitIterator`
- `ParentIterator`
- `RecursiveCachingIterator`
- `RecursiveCallbackFilterIterator`
- `RecursiveFilterIterator`
- `RecursiveIteratorIterator`
- `RecursiveTreeIterator`

Creating An `OuterIterator`

Like any interface in PHP, the `OuterIterator` interface can be implemented by any class. We could easily build our own example of an `OuterIterator`. However, for this demonstration, we will implement the `FilterIterator` abstract class which implements the `OuterIterator` interface.

`FilterIterator` already has the five main methods implemented as well as `getInnerIterator()`. It does, however, require that we implement the method `accept()`. `accept()` is a method that inspects the current value and determines whether or not it is acceptable. You can implement any rules in `accept()` that you wish as long as it returns a `true` or a `false`. Elements that `accept()` returns `true` are passed on to the calling looping structure. If `accept()` returns `false` then internally `next()` is called again on the inner iterator and then `accept()` is fired again. This continues until you either 1) receive a `true` from `accept()` or 2) the inner iterator’s `valid()` returns `false`.

We saw in the previous chapter how an iterator actually functions and in what order the methods are fired.

- Next
- Valid
- Current

In a `FilterIterator` you have an additional one. The order looks like this.

- Next
- Valid
- Accept
- Current

In the code that accompanies this book, you will find an example. We won't look at all of the sample code, but we will look at the `accept()` method and then how we implement our class.

```
public function accept()
{
        $current = $this->getInnerIterator()->current();

        return strpos($current, $this->filter) !== false;
}
```

Our `accept()` method is very simple. If the filter letter appears in the current value, we return `true`, otherwise, we return `false`. You can see that we are making a call to `getInnerIterator()`. We do this to get the current value of the inner iterator to do our comparisons on. This makes our `DwarfIterator` the `OuterIterator`; our inner `Iterator` is the `ArrayIterator` that contains the array of the names of the Dwarfs.

The implementation is equally simple.

```
$filter = substr((string) $input->getOption('filter'), 0, 1);

$dwarves = [
        'Grumpy ',
        'Happy ',
        'Sleepy ',
        'Bashful ',
        'Sneezy ',
        'Dopey ',
        'Doc ',
];

$dwarfIterator = new Iterators\Classes\DwarfIterator(
        new ArrayIterator($dwarves), $filter
);

foreach ($dwarfIterator as $thisDwarf) {
        $output->writeln($thisDwarf);
}
```

- We get our filter first. We only want a single character in this simple test so we filter it through `substr`.
- We create our list of the Seven Dwarfs. We are using the list from the 1937 animated Disney version. (Side note: Did you know that there were other lists? Wikipedia has an entire page dedicated to the different versions of the tale of Snow White and the names of the dwarfs in each).
- We create our `DwarfIterator`. Note that we do not simply pass in the array. An `OuterIterator` requires that there be an inner `Iterator`. We create an `ArrayIterator` which wraps our list of Dwarfs and that is our first parameter. The second parameter is the filter letter.
- We use a `foreach` to iterate over the list and output all that match the filter.

Now that you have seen the code, you can execute it like this:

```
$ ./console.sh outerIterator -f D
```

The `-f` parameter allows you to specify the filter to use. In the case above, it will only display the elements with `D` in the name. If you want to output the entire list, use `-f " "`. Each dwarf in the list ends in a space.

```
$ ./console.sh outerIterator -f " "
```

This is just one example of an `OuterIterator`. The `FilterIterator` is the easiest of the `OuterIterator` implementations to understand. As powerful as it is, you shouldn't stop exploring there. Many of `OuterIterators` built into the SPL will save you hours of implementing them yourself. Because they are implemented in the engine, your code will run faster. Iterators are a great example of how learning to use the SPL will change the way you code PHP.

Summary

We just scratched the surface of outer iterators in this chapter. With 13 concrete examples to play with, you already have a lot of great tools to work with and the example given shows how you can extend them to meet your own needs. Here are several take aways:

1. The `OuterIterator` interface allows us to build classes that can wrap other iterators;
2. To implement `OuterIterator`, you have to define the method `getInnerIterator()`;
3. The SPL contains 13 implementations of the `OuterIterator` interface;
4. The `FilterIterator` abstract class requires that we implement an `accept()` method;
5. The Disney version of "Snow White and the Seven Dwarfs" uses a non-standard list of names for the Dwarfs;
6. Learning the SPL will change the way you code PHP for the better.

Chapter 5 - The ArrayIterator

One of the most interesting iterators in the SPL is the `ArrayIterator`. PHP is well known for its array-handling functionality with more than 70 functions in the core language dedicated to processing them. The problem is that all of those functions are procedural in nature. Many developers these days want to work with arrays as first class citizens in our Object Oriented systems. PHP provides us with two ways to treat arrays as objects: the `ArrayObject` class and the `ArrayIterator` class.

`ArrayObject`

The simplest method of implementing an array as an object is to instantiate an `ArrayObject`. You can either hand your existing array into the constructor or you can create an empty `ArrayObject` and populate it manually.

```
$myArray = new ArrayObject(range(1, 100));

foreach ($myArray as $currentValue) {
     echo $currentValue . "\n";
}
```

The `ArrayObject` implements the following interfaces:

- `IteratorAggregate`
- `ArrayAccess`
- `Serializable`
- `Countable`

Since it implements `IteratorAggregate`, it is an Iterator in and of itself and as can be seen by the sample code above, and can be used in any situation that requires an `Iterator`, such as a `foreach`.

More importantly, `ArrayObject` encapsulates the functionality of many of the most commonly used array manipulation functions:

- `append()`
- `asort()`
- `count()`
- `ksort()`
- `natcasesort()`
- `natsort()`
- `serialize()`
- `uasort()`
- `uksort()`
- `unserialize()`

This allows us to manipulate our arrays in a much more natural Object Oriented environment.

```
$myArray = new ArrayObject(range(1, 100));

for ($i = 101; $i <= 125; $i++) {
     $myArray->append($i);
}

foreach ($myArray as $currentValue) {
     echo $currentValue . "\n";
}
```

In the above example, we create the same `ArrayObject` as before but then we use the `append()` function to add 25 more items to the list before printing them.

```
$myArray = new ArrayObject(range(1, 100));

$myArray->append(range(101, 125));

foreach ($myArray as $currentValue) {
     echo $currentValue . "\n";
}
```

Note that this will also work, but with very different results. Whatever you hand to append is appended to the bottom of the array as the next element. In the case of the example above, an array of numbers (101-125) would be added to the array as the next element. Thus, when the `foreach` hit that element, PHP would throw an error because it's can't `echo` and array.

`ArrayIterator`

In many cases an `ArrayObject` will do what you want to do. There is in fact, only a small difference between the `ArrayObject` class and the `ArrayIterator` class (other than the fact that `ArrayIterator is in the SPL and` ArrayObject `is not). The` ArrayIterator `class implements the` SeekableIterator `interface. Briefly, the` SeekableIterator `allows you to do more than move through the list from top to bottom, as most iterators do. The` SeekableInterface` allows you "random access" to the elements being iterated over.

The `ArrayIterator` implements the following interfaces:

- `ArrayAccess`
- `SeekableIterator`
- `Countable`
- `Serializable`

It is that `SeekableIterator` that makes all the differences as it gives us the `seek()` method.

```
$myArray = new ArrayIterator(range(100, 1, -1));

foreach ($myArray as $key => $currentValue) {
     echo $key . ':' .$currentValue . "\n";
}
```

```
$myArray->seek(10);

echo 'Element 10 contains the value ' .  $myArray->current() . "\n";
```

Note that `seek()` by itself only moves the internal pointer. If you want to use the value - in the example above, 90 is the value stores at position 10 of the array - then you have to use the `current()` method to retrieve it.

One problem that arises though is that `seek()` only takes integer values.

```
$myArray = new ArrayIterator(['good'=>'apple','better'=>'banana','best'=>'chocol\
ate']);

foreach($myArray as $key=>$currentValue) {
     echo $key . ":" .$currentValue . "\n";
}

$myArray->seek('better');

echo "Element 'better' contains the value " .  $myArray->current() . "\n";
```

The code above code will throw an error when it gets to the call to `seek()` because `seek()` expects an integer

```
$myArray = new ArrayIterator([
     'good'    => 'apple',
     'better' => 'banana',
     'best'    => 'chocolate',
]);

foreach ($myArray as $key => $currentValue) {
     echo $key . ":" . $currentValue . "\n";
}

$myArray->seek(2);

echo "Element 'better' contains the value " .  $myArray->current() . "\n";
```

This will work as `seek()` now has an integer but it is not really the effect we are looking for. To get the functionality we want, we will have to extend the `ArrayIterator` class and override the `seek()` method with our own.

```
class MyArrayIterator extends ArrayIterator
{
     public function seek($key)
     {
          $array = $this->getArrayCopy();

          $index = array_search(
               $key, $array_keys($array)
          );

          if ($index !== false) {
               parent::seek($index);
          }
     }
}

$myArray = new MyArrayIterator([
```

```
    'good'    => 'apple',
    'better' => 'banana',
    'best'    => 'chocolate',
]);

foreach ($myArray as $key => $currentValue) {
  echo $key . ":" . $currentValue . "\n";
}

$myArray->seek('better');

echo "Element 'better' contains the value " .  $myArray->current() . "\n";
```

Now our code works as we want it to. We can now pass in a string and if our `seek()` can find it as a key in the array, it calls the parent method with an integer. This is of course, an imperfect implementation. We do no type checking to make sure that we can get a valid result. If you were to use something like this in production, you would need to flesh it out a bit more.

Why Use An Iterator For An Array?

Now that we know about `ArrayObject` and `ArrayIterator`, the question that usually comes to mind is "Why not just use an array?" In many cases, there is no compelling reason to wrap your array in an object casing. We have discussed the most compelling case, making your arrays first class citizens in an Object Oriented system. For many developers, this is enough reason to do it right there. However, there is a technical reason. In Chapter 4, we discussed "Outer Iterators." Many times you will need your array to be processed by an outer iterator. In these cases, since outer iterators only work with iterators, you will have to wrap your array in either an `ArrayObject` or an `ArrayIterator` before you can hand it to your `OuterIterator`.

Offsets

If you have looked at the `ArrayIterator` manual page, you will notice the four methods beginning with 'offset'. The `ArrayAccess` interface is the one that gives us the methods to treat an object like an array. It defines these four methods.

`offsetExists()`

This method is fired returns `true` or `false`, based on whether the value passed in is a valid key or property in the current object. This is akin to the `valid()` method in the `Iterator` interface.

`offsetGet()`

This method takes a single parameter that is considered to be the key. Your code returns the value associated with this key.

`offsetSet()`

The opposite of `offsetGet`, this method takes a key and a value and your code will set the property associated with this key to the passed in value.

`offsetUnset()`

Called to remove a property from the object or collection, this method takes a simple parameter and your code is expected to remove the associated item from the collection.

Since `ArrayIterator` is a concrete implementation, we do not have to write any of this code, nor any of the code for the iterator methods. We can however, as demonstrated above, override any of the methods to extend the functionality.

Flags

Like many iterators defined in the SPL, the `ArrayIterator` has a series of flags that can be set to control its behavior. In the case of `ArrayIterator` there are only 2. The `setFlags()` manual page defines them for us.

- `ArrayIterator::STD_PROP_LIST` Properties of the object have their normal functionality when accessed as list (`var_dump()`, `foreach`, etc.).
- `ArrayIterator::ARRAY_AS_PROPS` Array indices can be accessed as properties in read/write.

The second flag is easily understood and demonstrated.

```
$myArray = new ArrayIterator([
     'good'    => 'apple',
     'better' => 'banana',
     'best'    => 'chocolate',
], ArrayIterator::ARRAY_AS_PROPS);

echo $myArray['good'] . "\n";
echo $myArray->best    . "\n";
```

This is basically syntactical sugar as the functionality does not change. But you can now access any element of your array as a property of the object.

The first one, however, is a bit of a conundrum. In previous versions of PHP, if you set the `STD_PROP_LIST` flag, properties you set on your object using the OOP syntax (`$myArray->awesome = 'peanutbutter'`) would be visible but properties you set using the array syntax (`$myArray['awesome'] = 'peanutbutter'`) would not. If the flag was not set, the reverse was true. A good example of this can be seen on the `ArrayObject` manual page. This flag worked as late as PHP 5.2. However sometime between 5.3 and 5.5, the behavior was changed so that PHP ignores the flag and properties set using both syntaxes are now always visible.

Summary

`ArrayObject` and the `ArrayInterface`are both interesting tools in your PHP toolbox. Make sure you familiarize yourself with them, how they work, and how you implement them so that when the time comes, you are ready to use them.

Both the `ArrayObject` and the `ArrayInterface` allow us to make arrays first class citizens in an Object Oriented program. We can control both the input and outputs of our arrays by overriding methods and inserting our own code.

The `SeekableIterator` gives us the power to pull random elements out of our iterator and not just iterate over it one element at a time.

Chapter 6 - …Of Inputs And Outputs

The SPL gives us great power. You know the saying, “With great power comes great amounts of flexibility and fun” (or something like that). Because the SPL gives us classes and interfaces, we are free to extend them, change them, make them operate in the way that we want instead of relying on the core contributors to figure out every possible scenario. Let’s take a look at two examples of this flexibility available to us, restricting input and re-jiggering output. Output first.

Modifying Output

Because `Iterator` is an interface that we can implement in our own classes, as long as we adhere to the API, we can do whatever we want in our code. We could easily implement a `UCFirstArray` iterator that modified the output of `current()` with the `ucfirst()` function before outputting it. That example, however, is boring and unimaginative. Instead, let’s consider the drinking-game-turned-developer-test ‘FizzBuzz’. Could FizzBuzz be implemented as an `Iterator`? The answer is, of course, yes or I wouldn’t have suggested it as an example.

For those not familiar with FizzBuzz, it is a common and fairly simple coding problem. Iterate over a list of numbers. For each number, apply the following rules:

- If the number is divisible by `3`, output “Fizz”.
- If the number is divisible by `5`, output “Buzz”.
- If the number is divisible both both `3` and `5`, output “FizzBuzz”
- Otherwise, output the number.

> The sample code that accompanies this book contains the full implementation. We won’t list it all here, just the important part. This code is from `classes/FizzBuzz.php`.

We put our logic in the `current()` method as that is what outputs the current value. Up to this point, we have always just output the current value of the item being iterated over. However, for this experiment, we want to modify the output instead of just passing it back out.

```
public function current()
{
        $number = $this->numbers[$this->pointer];

        $returnValue = '';
        $returnValue .= ($number % 3 === 0 ? 'Fizz' : '');
        $returnValue .= ($number % 5 === 0 ? 'Buzz' : '');

        if ($returnValue) {
                return $returnValue;
        }
```

```
        return $number;
}
```

As you can see, we have two ternary statements that determine the value to be returned. These two statements take the current value, test it, and create a new return value. If the number isn't divisible by `3` or `5` then the number is returned. It's that simple. We are now returning a value from `current()` that may not even exist in the underlying array.

Calling the program for the numbers `1-20` will give you this output.

```
$ ./console.sh fizzbuzz -m 20
  1
  2
  Fizz
  4
  Buzz
  Fizz
  7
  8
  Fizz
  Buzz
  11
  Fizz
  13
  14
  FizzBuzz
  16
  17
  Fizz
  19
  Buzz
```

In a previous chapter, we modified the content but returned a consistent data type. In this example we have shown that we don't even need to hold to that rule. We can code our iterators to return whatever we want, whether it is consistent or not.

That having been said, outside of very controlled situations like FizzBuzz, you probably *don't* want to be inconsistent in your return types. It's just not a good idea and can cause confusion among those trying to use your iterators in their own programs.

Restricting Inputs

We've seen how to take a list of numbers and output FizzBuzz based on them. Why though, do we let those non-Fizz and non-Buzz worthy numbers into our list in the first place? Let's openly discriminate against them and not let them into our list in the first place. Thankfully, PHP gives us this exact power by giving us control over our inputs.

The SPL contains a class that we can extend for our purposes, `ArrayIterator`. The `ArrayIterator` allows us to treat an object as an array. To work this magic, the `ArrayIterator` implements the following interfaces:

- `ArrayAccess`
- `SeekableIterator`
- `Countable`
- `Serializable`

To implement our filter, we need to override `offsetSet()`. This gives us the power to filter what goes in to our internal storage array.

```
class FizzBuzzFilter extends ArrayIterator
{
        public function offsetSet($key, $value)
        {
                $value = (int) $value;

                if ($value % 3 === 0 || $value % 5 === 0) {
                        parent::offsetSet($key, $value);
                }
        }
 }
```

It's wonderful how easy PHP makes these things. As you can see, all we have to do is override the `offsetSet()` method to apply our filter. If the number is a FizzBuzz number, we call the parent method with the parameters that were passed to us.

To implement this, we will copy `FizzBuzzCommand` and make `FizzBuzzFilteredCommand`, then change how the array is created to implement our new `FizzBuzzFilter`.

```
$max = max((int) $input->getOption('max'), 1);

$numbers = range(1, $max);

$fizzBuzzFiltered = new Iterators\Classes\FizzBuzzFilter();

foreach ($numbers as $thisNumber) {
        $fizzBuzzFiltered []= $thisNumber;
}

$fizzBuzz = new Iterators\Classes\FizzBuzz($fizzBuzzFiltered);

foreach ($fizzBuzz as $thisNumber) {
        $output->writeln("  " . $thisNumber);
}

$output->writeln("Done");
```

The key difference is how we handle the list of numbers. Previously we just created an array of numbers and filtered them on output. Now we still create that array, but we now load it into our `FizzBuzzFiltered` object to filter them on input.

```
$fizzBuzzFiltered = new Iterators\Classes\FizzBuzzFilter();

foreach ($numbers as $thisNumber) {
        $fizzBuzzFiltered []= $thisNumber;
}
```

Now in executing our new `FizzBuzzFiltered` we see that all the non-FizzBuzz worth numbers have been excluded from the group like so many high school nerds from a party at the quarterback's house.

```
$ ./console.sh fizzbuzzfiltered -m 20
  Fizz
  Buzz
  Fizz
  Fizz
```

```
  Buzz
  Fizz
  FizzBuzz
  Fizz
  Buzz
Done
```

No numbers, just Fizz and Buzz.

With that we have now implemented filters on both input and an output using Iterators. We've only scratched the surface of the power that iterators give us, and iterators are just a part of the power of the SPL.

Summary

1. Now that we've looked at how to extend iterators to incorporate your own ideas and code, it's up to you to find the proper uses for them. Remember, even though you've got a new tool in your toolbox, it is not appropriate for every situation. Practice using these new tools, play with them and the sample code we've worked with. Then, when you are comfortable with them, start rolling them into your projects. Here are a few of the important points from this chapter;
2. Iterators don't have to simply regurgitate the values stored in them. They can filter and manipulate the output to suite the business rules;
3. Iterators can be used to filter input so that unwanted data never gets into our collections to begin with;
4. Iterators give PHP developers a new power tool in their tool box.

Chapter 7 - The SeekableIterator Interface

In Chapter three we looked at the five main methods that `Iterator` has.

- `current()`
- `key()`
- `next()`
- `rewind()`
- `valid()`

These are fine for situations that call for a `foreach` (i.e. any situation where you want to start at the top of a list and iterate over it until you reach the bottom). That is what an iterator was designed for. However, needs change. What happens when you need to skip around in an iterator? There are times when you need to be able to move the pointer forward or backwards depending on what you are doing. Thankfully, PHP and the SPL have us covered with the `SeekableIterator` interface.

This interface defines a single method `seek()`. Because `SeekableIterator` is an interface and not a class, `seek()` can do whatever you want. It takes a single integer parameter. It is expected that you will implement code that will move the pointer to the new position, if it is valid. The reference code in the PHP manual shows that if the requested position is outside of the bounds of the current dataset, whatever that may be, that you throw an `OutOfBoundsException` exception.

```
public function seek($position)
{
     if (!isset($this->array[$position])) {
          throw new OutOfBoundsException(
               "invalid seek position (" . $position . ")"
          );
     }

     $this->position = $position;
}
```

It really can be that simple, or as complex as you like. You can put logic in your seek, just like you can in any of the other methods, to determine how you want it to operate. Take for instance our Seekable demo. It will allow you to iterate over the list of Fraggles we used earlier. You can see the complete code by downloading the repo; we won't reproduce it all here. Here is what the code does.

1. Instantiate a SeekableIterator
2. Iterate over the collection one at a time
3. Call `seek` to position the pointer for us
4. Output the current value

Simple enough. Here is the heart of the code.

```
for ($i = 0 ; $i < count($fraggleIterator); $i++) {
     $fraggleIterator->seek($i);

     echo $i . " : " . $fraggleIterator->current(). "\n";
}
```

For the most part, it looks pretty standard PHP code. However, because we are using a class that implements `SeekableIterator`, **and** it modifies the behavior of `seek()`, the results will not be what you expect. Go ahead, if you have it download and set up, run the command.

```
$ ./command seek
```

If you compare the output to the array, you will see that they are not the same. Here is what we've got going on in the `seek()`.

```
public function seek($position)
{
     $position = rand(0, count($this->data) - 1);

     if (!isset($this->data[$position])) {
          throw new OutOfBoundsException(
               "invalid seek position (" . $pointer . ")"
          );
     }

     $this->pointer = $position;
}
```

Regardless of what you pass in for the seek, we pick a random number between 0 and the length of the array and position the pointer there.

> I cannot stress this point enough **this code is for demonstration purposes only**. It was created to to show you that you do not have to follow the rules if you do not want to. It demonstrates that you can do processing and even filtering in the seek if you like. In most cases however, you **are** going to want to follow the rules. If you were to actually implement something like this in a production environment, well, it's what we call a *Career Limiting Move*.

Summary

The `SeekableIterator` interface allows you to build Iterators that give you more fine-grained control over your custom Iterators. the `seek()` method gives you one more point of control that you can use to customize the behavior of your Iterator and the data being iterated over.

- Defining `seek()` is as easy as any other method in any other Iterator.
- `seek()` takes exactly one parameter, the position your user is requesting to seek to.
- You can control exactly how your iterator seeks it's data.

Bonus

In the sample code, iterate over a list of Fraggles from the old HBO TV Show Fraggle Rock. There is one Fraggle missing from the list. There may be more than one, honestly; I

don’t know. There is one specific one that I know is missing. Email me at cal@calevans.com and tell me the name of the missing Fraggle. If you get it right, I will send you a free ticket to the Nomad PHP meeting of your choice.

Chapter 8 – Recursing The Iterator

Not all data is flat. When it is, it is easy to iterate over. A file, a stream, a single dimension array, these are all easy to iterate over because they are flat. Data however, like life, is not always flat. For those times, PHP gives us a series of eight Recursive Iterators.

- `RecursiveArrayIterator`
- `RecursiveCachingIterator`
- `RecursiveCallbackFilterIterator`
- `RecursiveDirectoryIterator`
- `RecursiveFilterIterator`
- `RecursiveRegexIterator`
- `RecursiveTreeIterator`
- `RecursiveIteratorIterator`

In concept, Recursive Iterators are simple. They help you iterate over non-flat data. For example, the `RecursiveArrayIterator` can be used to iterate over an array containing arrays.

New Methods

Recursive iterators work by implementing the `RecursiveIterator` interface. The `RecursiveIterator` interface defines two additional methods that the recursive iterators have to implement.

- `hasChildren()`
- `getChildren()`

The first is called to see if the current item actually has children; the second is called if children are found.

`RecursiveIteratorIterator`

More jokes have been made about this one class than any other that I know about in PHP. Drinking games have been invented to play at conferences featuring a talk about Iterators and `RecursiveIteratorIterator` usually puts the participants under the table. `RecursiveIteratorIterator` is actually an important class to understand, though. By themselves, recursive iterators understand how to recurse over a given collection. However, something else has to control the entire process. `RecursiveIteratorIterator` is that class. It is designed, not to iterate over a specific type of data as the other recursive iterators are, but to iterate over a class implementing the `RecursiveIterator` interface.

We will dive deeper into the use of the `RecursiveIteratorIterator` in the next chapter and actually show an example of how it is used. For now, it is enough to know that it is

there, and no matter what you think of the name, you will eventually come to love it.

The Most Common Use Case

If you have done much PHP development at all, you have run into a situation where you need to iterate over a multi-dimensional array. Most of us have PHP code we can quickly adapt to the task by copying something from a previous project. PHP however, gives us the tools to easily do this with the `RecursiveArrayIterator`. As discussed above, though, `RecursiveArrayIterator` alone won't do the job. `RecursiveArrayIterator` understands how to process an array. It also knows how to tell the level above it that the current element is not a single element but a set of elements. (`hasChildren()`) However, it doesn't know how to properly process these sets of elements, that's not it's job. For that job, we use the `RecursiveIteratorIterator` discussed above. `RecursiveIteratorIterator` understands how to properly process a collection that is made up of both single elements and other collections. In the next chapter we will get into a more detailed explanation of how this works, but for now, let's look at this piece of sample code.

To run the sample code, use this command.

```
$ ./console.sh recursive
```

You will notice that the first thing it does is `print_r()` the array we are about to iterate over. You can see that the array contains both arrays and single items. Next, we output the array keys and elements one at a time. Here is the code we use to do this.

```
$characters = include '../examples/southparkCharacters.php';

$output->writeln("Source Array:");
print_r($characters);

$output->writeln("Iterator Output");
$iterator = new \RecursiveIteratorIterator(new \RecursiveArrayIterator($characte\
rs));

foreach ($iterator as $key => $value) {
     $output->writeln($key . " : " . $value);;
}
```

Notice that the `foreach` that does the output is a very simple `foreach`. No extra processing for recursing through the child elements. Simple and straightforward, thanks to the combination of `RecursiveArrayIterator` and `RecursiveIteratorIterator`. Thanks to the SPL, all of the work is now handled in the engine and now in your userland code.

One thing you will notice is that the keys for the sub-arrays – in the case of our sample data, 'main', 'recurring', and 'seasonal' – are no where to be found. `$key` always refers to the current element key and the current element is never the sub-array itself, it is the element. This is because we are actually iterating over `RecursiveIteratorIterator`, not `RecursiveArrayIterator` or the array. `RecursiveIteratorIterator` is getting the data from `RecursiveArrayIterator` and processing it. It leaves off the key from these elements because it doesn't actually process the elements themselves.

We could of course subclass `RecursiveIteratorIterator` and add a method `getParentKey()` that would return this information for us. The beauty of PHP and the way

that the SPL is designed for us, it is a framework that we can build upon, not the final solution.

Summary

Recursive iterators are not natively intuitive. Most PHP developers, when they first encounter them, expect to be able to iterate over complex sets of data easily. It is only when you begin to understand that they are a piece of the solution, not the entire solution, that things begin to fall into place and they begin to understand the power at their fingertips.

- Recursive iterators by themselves won't solve your problem; you have to wrap them in the `RecursiveIteratorIterator` to be able to easily recursive over complex data sets.
- You are not limited to the recursive iterators built into PHP. You can subclass them and extend them or you can build your own using the `RecursiveIterator` interface.
- Once you understand the power available to you, you won't look at iterating over complex datasets the same way again.

Chapter 9 - Iterating The File System

Every programmer I know has written code to iterate over the filesystem of a computer and print out the files found. Regardless of the language, at some point we all do this. It's also a great introduction to recursion because it is an easy problem to understand.

`DirectoryIterator`

In PHP, we have several different ways we can accomplish this task. The ones we will look at in this chapter are Iterators. The first of these is the `DirectoryIterator`. You can open the `DirectoryCommand` file if you've downloaded the source code and see the entire command. For our discussion here though, we are only going to look at the main method of that object, `method1()`.

```
protected function printTree($dir, $padLength = 0)
{
     $tree = new DirectoryIterator($dir);

     foreach($tree as $node) {

          // Skip the dot directories

          if ($node->isDot()) {
               continue;
          }

          // if this is a directory AND we have permission to read it,
          // iterator over the contents of the directory

          if ($node->isDir() && $node->isReadable()) {
               echo str_repeat(' ', $padLength * 2) . $value . "\n";

               $this->printTree($dir . '/' . $value, $padLength + 1);
          }

          // if this is a file, print the filename

          if ($node->isFile()) {
               echo str_repeat(' ', $padLength * 2) . $value . "\n";
          }
     }
}
```

As you can see, this is just about the simplest version of a recursive directory iterator you can write in any language. Not quite **the** simplest, but real close.

For each directory passed in, we create a `DirectoryIterator`. Then we iterate over it applying the following rules.

- Ignore dots. We don't care about `.` or `..`

- If it is a directory **and** we have permission to read it, then have the method call itself with the new information so that we can iterate over this new directory.
- If it is neither of those then it's a file; print out the name.

`FileSystemIterator`

Previously we introduced the `FilesystemIterator`. Let's look a little deeper into what this class gives us. In the sample code above, we iterate over each entry in each directory. We check to see if a directory is readable before recursing into it. This is a simple enough check because the `DirectoryIterator` simply checks the properties of the entity for us. In the `foreach`, `$value` is a string, the name of the file. You will notice that in the code above, we append the directory name to the file for output. That is because `$value` is **only** the file name and extension. That changes with `FilesystemIterator`. `FilesystemIterator` is a subclass of `DirectoryIterator`. It adds several new methods and implements the `SeekableIterator` interface we discussed in the previous chapter.

Most importantly though, when you iterate over a `FilesystemIterator`, you do not get a string in `$value`, you get a `SplFileInfo` object. You could easily switch out a `FilesystemIterator` for the `DirectoryIterator` above because a `SplFileInfo` implements the `__toString()` method and returns the full pathname and file name of the entity.

Where you would run into a problem is if you tried to instantiate a `FilesystemIterator` for a directory that we do not have read permissions on. We solve this by wrapping our creation of the `FilesystemIterator` in a `try/catch`. If creating it fails, we just return and move on to the next item.

```
try {
    $tree = new FilesystemIterator($dir);
} catch (UnexpectedValueException $e) {
    return;
}

foreach($tree as $node) {
    if ($node->isDir()) {
        echo str_repeat(' ', $padLength*2) . $node . "\n";

        $this->printTree($node->getPathname(), $padLength + 1);
    }

    echo str_repeat(' ', $padLength * 2) . $node->getPathname() . "\n";
}
```

Now, our code will operate as expected.

Flags

`FilesystemIterator` has several flags that control how it works. Here are the ones that are most commonly used.

`CURRENT_AS_PATHNAME`

If this flag is set, `current()` will return the path name, including the file name and extension of the current file will be returned.

CURRENT_AS_FILEINFO

If this flag is set explicitly, or if none of the `CURRENT_AS_*` are set, calling `current()` will return a `SplFileInfo` object. **This is the default behavior.**

CURRENT_AS_SELF

This will cause the `current()` method to return a reference to the `FilesystemIterator` object itself. If this flag is set, our sample code will operate the same as if the default `CURRENT_AS_FILEINFO` was set. This is because `FilesystemIterator` and `SplFileInfo` both have the method names we are calling, specifically, `isDir()` and `getPathname()`.

KEY_AS_PATHNAME

This is the default behavior. calling `key()` will return you the full path and file name of the file.

KEY_AS_FILENAME

If you set this flag, you will only get the file name and extension, not the full path.

FOLLOW_SYMLINKS

This flag is only useful if you are using a `RecursiveDirectoryIterator`, a subclass of `FilesystemIterator`. If you are working with a `RecursiveDirectoryIterator` and you set this flag, then `hasChildren()` will follow symlinks. If you are not using a `RecursiveDirectoryIterator` or a subclass of it, this flag has no affect.

NEW_CURRENT_AND_KEY

This is an alias for setting both `FilesystemIterator::KEY_AS_FILENAME` and `FilesystemIterator::CURRENT_AS_FILEINFO`. **This is the default behavior.**

SKIP_DOTS

Setting this flag tells the `FilesystemIterator` to ignore directories named `.` and `..`. Almost every time you iterate over a directory, you will want to ignore these directories. **This is default behavior.**

UNIX_PATHS

This flag tells the `FilesystemIterator` to use Unix style paths with a `\` as the separator, regardless of what OS you are running the script on.

Bitmasks

There are three other flags that you can use with the `FilesystemIterator`. These flags, however, are used as filters for the results of `getFlags()`.

- `CURRENT_MODE_MASK`
- `KEY_MODE_MASK`

> For a introduction to bit math, and examples of how to use these flags, check out **Appendix A**.

`RecursiveDirectoryIterator`

In the first section of this chapter, we examined code that recursively iterated over a directory. It wasn't very complex code, but now that we know about the

`FileSystemIterator`, we can look at the `RecursiveDirectoryIterator` and make the code even simpler.

In the last chapter, we discussed the `RecursiveIteratorIterator`. Now let's put it to good use. Let's build a recursive directory iterator that is even simpler that the example above.

```
protected function printTree($dir, $padLength = 0)
{
     $objects = new RecursiveIteratorIterator(
          new RecursiveDirectoryIterator($dir),
          RecursiveIteratorIterator::SELF_FIRST
     );

     try {
          foreach ($objects as $name => $object){
               echo $name . "\n";
          }

     } catch (UnexpectedValueException $e) {
          // ignore this entry
     }
}
```

By chaining two iterators together, we now have a single, no-recursive, `foreach` loop that we can call to output the contents of our directory and it's subdirectories.

In this case we do have to write it in a `try/catch`. `RecursiveDirectoryIterator` does not directly subclass `DirectoryIterator`. Instead, it extends `FilesystemIterator`. `FilesystemIterator`, which we will look at below, outputs a `SplFileInfo` instead of just a strong string with the name of the entity. Creating a `SplFileInfo` requires that PHP be able to read the attributes of the entity to populate the attributes of the `SplFileInfo` object. If the account that is running the script does not have these permissions, an exception is thrown. For simplicity sake, we just ignore these entities. Depending on the task you were trying to accomplish, you might want to do additional processing in the `catch`. Possibly log the entities to which you did not have access.

Summary

Building on what we learned about recursive iterators, we've now looked at three ways to iterate over a filesystem. The hard way, the easier way, and the easiest way. This doesn't mean that using `RecursiveDirectoryIterator` and `RecursiveIteratorIterator` is always the best way to do it. As always, know all the ways to accomplish a task so that when you are presented with the task, you can choose the proper one for the situation.

- `DirectoryIterator` is a useful tool but requires you to write more code than other options.
- `FileSystemIterator` extends `DirectoryIterator` and has a lot of power built into it. You have to master a bit of bit math to get the full power, but it's worth the effort.
- `RecursiveDirectoryIterator`, when combined with `RecursiveIteratorIterator` is the dead simplest way to iterate over a portion of your filesystem.

Chapter 10 - The Caching Iterator

`CachingIterator` is the last iterator we will look at in this book, we've saved an interesting one for last.

The most common use for the `CachingIterator` is looking ahead at the next element without moving the pointer. In discussing this class with several people - including at least one core developer - I was told that the `CachingIterator` really should be called the `LookAheadIterator` as that is really what it was designed to be. Below we look at a simple example to help illustrate the "look ahead" feature. Once we master it, we will look at some of the more edge-case uses of `CachingIterator`.

`CachingIterator` And "Look Ahead"

Let's look at a simple example. Let's use the `CachingIterator` to create a Comma Separated Value (CSV) list of the Dwarfs we used earlier. Yes, this example could easily be built with `implode()`. However, it's a good example of needing to treat an element differently than the others as the last element does not get a comma appended to it.

```
protected function execute(InputInterface $input, OutputInterface $output)
{
    $dwarves = [
        'Grumpy',
        'Happy',
        'Sleepy',
        'Bashful',
        'Sneezy',
        'Dopey',
        'Doc',
    ];

    $dwarfIterator = new ArrayIterator($dwarves);

    $cachingDwarfIterator = new CachingIterator($dwarfIterator);

    $dwarfListOutput = '';

    foreach ($cachingDwarfIterator as $thisDwarf) {
        $dwarfListOutput .= $thisDwarf;

        if ($cachingDwarfIterator->hasNext()) {
            $dwarfListOutput .= ',';
        }

    }

    $output->writeln($dwarfListOutput);
    $output->writeln("Done");
}
```

As you can see above, the `CachingIterator` comes into play inside the `foreach` loop. We fire the `hasNext()` method to determine whether or not we should add a comma to the output.

That is the heart of the `CachingIterator`, the `hasNext()` method. It literally tells us if there is another value for us to look at. If you master that one concept then you have mastered most of the uses of `CachingIterator`.

Since `CachingIterator` implements `OuterIterator`, `ArrayAccess`, and `Countable`, you get all the methods we have discussed in previous chapters. `hasnext()` and `getInnerIterator()` are the new methods we have to play with, the latter only when `FULL_CACHE` is set. More about that in a bit.

Flags

Like other iterators, `CachingIterator` has several flags that you can pass into the constructor to change it's behavior. You can also use `setFlags()` method to set them after the object has been instantiated.

Handling Casting The Iterator To A String

These flags are not mutually exclusive. You can set several using the bitwise or operator `||`. Doing do however, may not give you the results you are expecting. For example, if you set both `TOSTRING_USE_CURRENT` and `TOSTRING_USE_INNER` PHP will ignore the first and use the second.

`CALL_TOSTRING`

Exactly what it sounds like. When you retrieve the value of an element, the `CachingIterator` will pass it through it's `__toString()` before returning it. **This is the default behavior.**

`TOSTRING_USE_KEY`

If this flag is set, casting the iterator to a string will give you a string containing the value of the current **key**. It uses `key()`.

`TOSTRING_USE_CURRENT`

If this flag is set, casting the iterator to a string will give you a string containing the value of the current **value**. It uses `current()`.

`TOSTRING_USE_INNER`

If this flag is set, casting the iterator to a string will call the inner iterator's `__toString()` and return its **value**. You must insure that the the inner iterator implements the `__toString()` method.

Other Flags

`CATCH_GET_CHILD`

This flag comes to us from the `RecursiveIterator`. It will capture any exceptions thrown when you call `getChildren()` because there are no children.

FULL_CACHE

This one is a little tricky. If you set `FULL_CACHE` then as you iterate over the inner iterator, the `CachingIterator` will cache it for later use. The confusing part is that if you decide to make changes to your data, things get weird in a hurry. The changes you make are made on the cached version, not the original. They will also be erased if you `rewind()` your iterator. The `FULL_CACHE` flag is the only flag that gives you access to the `getInnerIterator()` method. If you call `getInnerIterator()` on a `CachingIterator` without it set, it will throw an exception.

Let's take a look at the sample code to demonstrate.

```
$dwarfIterator = new ArrayIterator($dwarves);

$cachingDwarfIterator = new CustomCachingIterator($dwarfIterator,::FULL_CACHE);

// Load the cache

while ($cachingDwarfIterator->valid()) {
     $cachingDwarfIterator->next();
}

$cachingDwarfIterator->removeBashful();
```

For our example, we are going to remove one of the dwarves from the list by calling `removeBashful()`. However, before we do this, we want to load the cache. By calling an empty `foreach`, we force the `CachingIterator` to iterate over the entire list and store all the current keys and values in the cache. Then, we call `removeBashful()` to remove an element from the array.

```
$output->writeln(' ');
$output->writeln(' Cached');

foreach ($cachingDwarfIterator->getCache() as $key=>$thisDwarf) {
     $output->writeln($key . " : " . $thisDwarf);
}

$output->writeln(' ');
$output->writeln(' Original');

foreach ($cachingDwarfIterator as $key => $thisDwarf) {
     $output->writeln($key . " - " . $thisDwarf);
}
```

In the above code, you will see that the Cached version is the version that is modified by the `offsetUnset()` call. This also goes for elements that you change.

```
$output->writeln(' ');
$output->writeln(' Modify an element');

$cachingDwarfIterator[5] = 'Surley';

$output->writeln(' ');
$output->writeln(' Cached');

foreach ($cachingDwarfIterator->getCache() as $key => $thisDwarf) {
     $output->writeln($key . " : " . $thisDwarf);
}
```

```
$output->writeln(' ');
$output->writeln(' Original');

foreach ($cachingDwarfIterator as $key => $thisDwarf) {
     $output->writeln($key . " : " . $thisDwarf);
}
```

This code shows that modifying an element changes the cached version, but not the original as well. Not shown here but found in the sample code is the class definition for `CustomCachingIterator` used above.

Summary

The `CachingIterator` is a powerful tool in your toolbox as long as you understand how to work with it. On it's surface, it is great for "look ahead" operations. The most common example of using it in this way is building HTMl for navigation where the last element gets handled differently. The example we used was building a CSV list of the dwarves. There are better ways to build the CSV but it is an easily understandable example of how and when to use the `CachingIterator`'s lookahead. Modifying the content of your `CachingIterator` however, may produce odd results, based on which flags you have set and what you are doing.

Once you get into the flags, things can easily start to go sideways. The `TOSTRING` flags are fairly straightforward. However, once you get into the `FULL_CACHE` flag, you have to remember which version of the data you want to iterate over. That having been said, once you understand how to work with it, you will find more and more uses for it.

- The most common use of `CachingIterator` is for it's "look ahead" capability.
- The several of the flags for `CachingIterator` determine how it handles being cast as a string.
- The `FULL_CACHE` flag is the most complex of the flags but give developers a new and powerful tool.

Chapter 11 - Generators

What Are Generators?

Starting with PHP 5.5 we got yet another way to iterate over collections; “Generators”. While Generators are new, they don’t really give us any new functionality. Instead, they give us a shortcut for iterating over something…quite literally, anything.

On the surface, generators look at lot like normal PHP functions. You define them as a function, but with one very important difference, you use the `yield` keyword to return data out of your generator. Once PHP sees that your method has a `yield` in it, it will be treated as a generator. Let’s look at a very simple example.

```
$myGen = myGenerator();

foreach ($myGen as $currentValue) {
     echo $currentValue . "\n";
}

exit;

function myGenerator()
{
     for ($i = 1; $i <= 10; $i++) {
          yield $i;
     }
}
```

This is the quintessential generator example. There is very little advantage to writing this generator. We look at it here just to show you the structure of a generator.

Line 1: This line looks like normal PHP. If our `myGenerator()` function had a `return` instead of a `yield` then we would know exactly what was going on here, although we would be a little confused. a `return` in place of the current `yield` would mean that the counter would never go above 1. Once our method hit `return` it would return and `$i` would go out of scope and be flagged for garbage collection.

Line 4: If myGenerator was an actual function and not a generator, this would throw a fatal error. PHP needs something that can be iterated over in a foreach.

Line 9: Again, this looks like a normal PHP function definition. We could even pass in a parameter if we wanted to. However, since `myGenerator()` has a `yield` in it, PHP knows to treat it differently.

Line 11: The heart of our generator is a loop to count form `1` to `10`.

Line 12: This is where the magic happens. Instead of `return $i;` we have `yield $i;`.

You can copy the above code, paste it into a file, save it as `genTest.php`, and then run it from the command line.

```
$ php myGen.php
```

It will count from `1` to `10`. I know, amazing, right?

So what is really happening here? As stated earlier, when PHP sees the `yield` it turns the method into an object that is an instance of the `Generator` class. The `Generator` class implements the `Iterator` interface. PHP has wrapped your code in just enough of an object to make an Iterator. That is why you can use it inside a `foreach`.

This leads us to the next interesting fact. Because your function is now a Generator, and you don't `return` from it you `yield` results, the execution of the function doesn't actually end when it hits the yield, it just pauses. This means that in our very simple example above, at line 4, `$i` is still valid, although it's not in scope. The function has not ended so any and all variables defined to that point as still there and still hold a value, if they have been assigned. Things just got interesting. Now I have a lot of the benefits of an `Iterator` object, but I haven't had to go through the pains of creating a class, defining properties that need to be persistent, etc. Generators handle that for me.

Whether it is a variable, a stream, a database connection, anything, whatever you have defined in your generator lives until the generator is done.

A More Complete Example

Now that we've seen a simple example, let's revisit the example we created in chapter 3.

In chapter 3, we built our first Iterator. We iterated over a list of the most recent committers to the PHP core repository on Github. To do this, we had to create a class that implemented an interface. We had to write code for the five methods defined in the interface write code for five different methods in that class. All told, we had to write more than `150` lines of code (depending on which version actually ships) to be able to display that list. What if there were an easier way? Now, with generators, we can do all of this in a single function.

Welcome to generators!

You can see the example in GeneratorCommand.php. I won't pad the book by pasting it all in here. I will however, paste the main function in so that we can discuss it.

```
protected function gatherCommits($gatheredComits, $verbosity)
{
    $committers = [];

    foreach ($gatheredComits as $thisCommit) {
        $authorId = $thisCommit['author']['id'];

        if (!isset($committers[$authorId])) {
            $url = 'https://api.github.com/user/' . $authorId;

            $committers[$authorId] = Guzzle::get($url)->json();
        }

        $payload = [
            'commit'     => $thisCommit,
            'committer' => $committers[$authorId]
        ];

        if ($verbosity === OutputInterface::VERBOSITY_VERBOSE) {
```

```
            echo "before yield\n";
        }

        yield $payload;

        if ($verbosity === OutputInterface::VERBOSITY_VERBOSE) {
            echo "after yield\n";
        }
    }
}
```

We have significantly cut down our line count now.

Notice, there are no object properties being used. The `$committers` array is defined within the scope of `gatherCommits()`. When `gatherCommits()` completes, it will no longer exist. This example is a little more complex than the initial example, but not by much. We used Guzzle to pull the commits in and we pass them into `gatherCommits()` as a parameter.

Using the `fornext()` we spin through the array of commits and build a $payload `for each commit. $payload is an associative array with 2 indices, "commit" and "committer". Once $payload is built, we yield it back to the calling routine. At this point, the gatherCommits() pauses. The yield hands control of execution back up the line. The next time the fornext calls the generator, it picks up immediately after the yield statement. I've thrown in a couple of echo` statements to show you the workflow.

To see the Generator object, run the sample code with `DEBUG` verbosity.

```
$ ./console generator -vvv
```

To see “before yield” and “after yield” echo statements, use the `VERBOSE` verbosity level.

```
$ ./console generator -v
```

Of course without and `-v` flags, you will get just the list of people you need to find and say thank you to for contributing to PHP.

Summary

Generators are a powerful and easy shortcut for many tasks you would normally write your own `Iterator`. Once you understand them (and they are not difficult to understand) you will start seeing places where you can use them. Then you’ll look back in six months and wonder how you coded without them.

- You define a generator like you define a function, only you use a `yield` statement instead of a `return` statement.
- Unlike a `return` statement a `yield` only pauses execution of the generator.
- Variables defined inside of your generator live within that scope until it finishes iterating over whatever it is you are iterating over.

Appendix A - A Bit Math Primer

In “Iterating the File System” we discussed the `FilesystemIterator`. That class has flags like a lot of other SPL classes do that have to be `OR`ed together. Unlike a lot of classes though, `FilesystemIterator`provides three masks that can be used to view portions of the flags while ignoring the other parts. To understand how all of this works, let’s talk a little about bit math.

Starting Small

Before we dive into the `FilesystemIterator`, though, let’s cover the basics. Let’s start with a four bit number, `0000`.

There are two operations we are going to cover in this primer, `AND` and `OR`. We will briefly touch on `XOR` but for the most part, you won’t be using that. For our purposes, `OR` is how you set a bit, and `AND` is how you check the value of a bit. This is a gross over simplification, but is serves our purposes.

In bit math, each bit has a specific value. In our four bit example, the bits are valued like this.

```
Bit Number : 1  2  3  4
===========:===========
Value      : 1  2  4  8
```

In a 4 bit number like this, the largest value that can be stored is `15` (`8` + `4` + `2` + `1`).

When our number has the value of 6, the bits look like this.

```
Bit Number : 4  3  2  1
===========:===========
Value      : 8  4  2  1
===========:===========
             0  1  1  0
```

The bits for the values `2` and `4` are “set” (have a value of `1`). The rest of the bits are “clear” (have a value of `0`).

Many times in programming (although not a lot in web based programming) we use a technique called “bit packing”. Bit packing is nothing more than using each of the bits in a number to represent a boolean state or a “flag”. If a bit is set, then the flag is set. If a bit is clear the the flag is clear. The individual bits are assigned a meaning by the programmer. In the four bit example above, four flags can be stored. In a normal eight bit number, eight flags can be stored in a single byte.

To set a bit, we `OR` it with a number. For example, if we want to set the second bit in our 4 bit number above, we would `OR` `0000` and `0100`. In PHP, we write this like this.

```
$flags = 0;
$flags = $flags | 2;
```

```
echo $flags;
```

That would set the second bit and now `$flags` has a value of `2`.

To clear the flag, we use the `XOR` operator (`^`).

```
$flags = 2;
$flags = $flags ^ 2;

echo $flags;
```

Now you can see the basics. If you want to see it in operation, here is a quick piece of sample code you can paste into a file and run.

```
$thisFlag = 2;
$flags    = 0;

echo str_pad(decbin($flags), 4, '0', STR_PAD_LEFT) . "\n";

$flags = $flags | $thisFlag;

echo str_pad(decbin($flags), 4, '0', STR_PAD_LEFT) . "\n";

$flags = $flags ^ $thisFlag;

echo str_pad(decbin($flags), 4, '0', STR_PAD_LEFT) . "\n";
```

`decbin()` converts a decimal value into binary, or bits, and `str_pad()` just pads them out so that all the numbers have all 4 bits showing.

The first `echo` just shows you what a `0` value looks like. Then we set the `2` flag and echo it again. Then we clear the `2` flag, and echo it once more.

Try different values for `$thisFlag` and look at the results.

N> ## `AND` vs. `|` N> the words `AND` and `OR` seem analogous to the operators `&` and `|`. However, they are not. The difference is "precedence". They are not always interchangeable. It is important to understand this. In almost every case, when you are doing bit math, you will want to use the symbols and not the words.

Bigger And Better

Now that we have looked at the basic operations, we can move on to the `FilesystemIterator`. `FilesystemIterator` lists the following flags. These are defined as constants and are assigned the following values.

- `CURRENT_AS_FILEINFO` = `0`
- `KEY_AS_PATHNAME` = `0`
- `CURRENT_AS_SELF` = `16`
- `CURRENT_AS_PATHNAME` = `32`
- `KEY_AS_FILENAME` = `256`
- `NEW_CURRENT_AND_KEY` = `256`
- `FOLLOW_SYMLINKS` = `512`
- `SKIP_DOTS` = `4096`
- `UNIX_PATHS` = `8192`

The flags used by the `FileSystemIterator` are held in the first 14 bits of an integer. Depending on the system you are running on, the total number of bits in an integer are either 32 or 64. The value of the flags we are concerned with can range from `0 - 8192`.

As we discussed in the section above, each bit represents a number. Here is a list of the bits and their associated values for the flags that `FileSystemIterator` uses.

1. 1
2. 2
3. 4
4. 8
5. 16
6. 32
7. 64
8. 128
9. 256
10. 512
11. 1024
12. 2048
13. 4096
14. 8192

Given this chart, if the third bit is set the value of the integer is `4`. If we set the 3rd bit and the 5th bit – `4` and `16` - then the value of the integer is `20`. To find the value of the integer, you simply add up the values of all the bits set to `1`.

To set a single bit in an integer, you `OR` the current value with the value of the bit you want to set. Optionally, you can use `|`.

Now that you know the basics, let's look at it visually. If we want to see how an integer breaks down, we can use a little tool in the sample code called bit math. All it does it break an integer down into its individual bits. This output is set up to display the 14 bits used by `FileSystemIterator`.

```
$ ./console.sh bitmath -u 1024
```

This outputs a simple table of the bits and their values, and then puts a `1` or a `0` in each box depending on whether it was turned on.

You can see from the output of the command above, `1024` only has a single bit set, the 11th bit.

Experiment a little, try different numbers. See what flags are set when you use `3`, `12`, `42`. Add the values for each set bit up and confirm the math.

Now that you are comfortable with a single number, let's pretend we are setting flags. At the start of this section, we listed out the flags that `FileSystemIterator` uses. We have also discussed that to set 2 flags, we `OR` the values together. so if we wanted to set both `CURRENT_AS_PATHNAME` and `KEY_AS_FILENAME` we would have to `OR` `32` and `256`. You can do the math in your head if you like, I'd rather just use a little Bash magic.

```
$ ./console.sh bitmath -u $((32|256))
```

You can see that the resulting number is 288, the sum of the two numbers. That is not always the case in bit math. However, since the flags we are working with are all set by single bits, it will be true for the `FileSystemIterator`.

The final thing you need to know about bit math is how we mask out a bit or set of bits to see just their value. There are times when you do not want to see the entire value of the number, you only want to see the value of certain bits. To accomplish this, we use the `AND` operator or alternately the `&`.

For example, if we run our bit math script for the number `42`, we will see that `2`, `8`, `32` are set. We can see that visually with the command

```
$ ./console.sh bitmath -u 42
```

However, what if the 6th bit – value `32` – is a flag and we are interested in seeing if it is set. To do this, we use the `AND` operator. To see it visually in our sample code, use the following command.

```
$ ./console.sh bitmath -u $((32&42))
```

Notice that now the **only** bit that is set is the `16` bit. The `&` filtered out everything else. Even though we know that `2`, `8`, and `32` are all set to make up `42` (add them up if you need to) by masking `42` with `32`, we see only see `32`.

We can also mask a series of bits. For example, if we want to see the `2`, `4` and `8` bits of `42`, we could use this command.

```
$ ./console.sh bitmath -u $((14&42))
```

Now our program ignores the `32`, because it is not the `2`, `4`, or `8`.

Practically Speaking

In your PHP code you won't be using our bit math script to see flags. You will be using the operators directly.

> Honestly, bit math was a big technique back when RAM and hard-disks were expensive. These days it is not necessary to pack and unpack bits like this unless you just really want to. In the chapter on the `FileSystemIterator` we see that they are using masks to show us things, so we will use that as the example here.

To use masking to see what flags are set on the `FileSystemIterator`, we use statements like this.

```
$tree = new FilesystemIterator('/tmp');

echo "Flags        :" . $tree->getFlags() . "\n";
echo "Current Mode :" . ($tree->getFlags() & FileSystemIterator::CURRENT_MODE_MA\
SK) . "\n";
echo "Key Mode     :" . ($tree->getFlags() & FileSystemIterator::KEY_MODE_MASK) \
. "\n";
echo "Other Mode   :" . ($tree->getFlags() & FileSystemIterator::OTHER_MODE_MASK\
) . "\n";
```

If you put the code above in a script named `myMaskTest.php`, save it and run it, you will get this output.

```
$ php myMaskTest.php

Flags        :4096
Current Mode :0
Key Mode     :0
Other Mode   :4096
```

Note that without any flags set, the default value for your flags is set to `4096`. This is because the default flags that are set are as follows.

- `CURRENT_AS_FILEINFO` = `0`
- `KEY_AS_PATHNAME` = `0`
- `SKIP_DOTS` = `4096`

If you add the values up (`0` + `0` + `4096`) you get the value displayed above.

`CURRENT_MODE_MASK` and `KEY_MODE_MASK` both return `0` when you mask them because the flags that are set for them have a value of `0`. (No flags set is the default) `OTHER_MODE_MASK` returns an integer of `4096` because the value of the default other flag - `SKIP_DOTS` - is `4096`.

You can experiment with the above code. Try replacing the first line with one of the following.

```
$tree = new FilesystemIterator(
     '/tmp', FileSystemIterator::CURRENT_AS_PATHNAME
);
```

or

```
$tree = new FilesystemIterator(
     '/tmp',
     FileSystemIterator::CURRENT_AS_PATHNAME |
     FileSystemIterator::KEY_AS_FILENAME
);
```

or

```
$thisDir = new FilesystemIterator(
     '/tmp',
     FileSystemIterator::CURRENT_AS_PATHNAME |
     FileSystemIterator::KEY_AS_FILENAME |
     FileSystemIterator::UNIX_PATHS
);
```

This final example will output the following.

```
$ php myMaskTest.php

Flags        :12576
Current Mode :32
Key Mode     :256
Other Mode   :12288
```

Now, you can take the value of Flags – `12576` – and the value of the `OTHER_MODE_MASK`, `12288`, and you can use our bit math script to see which bits are actually set.

```
$ ./console.sh bitmath -u $((12576&12288))
```

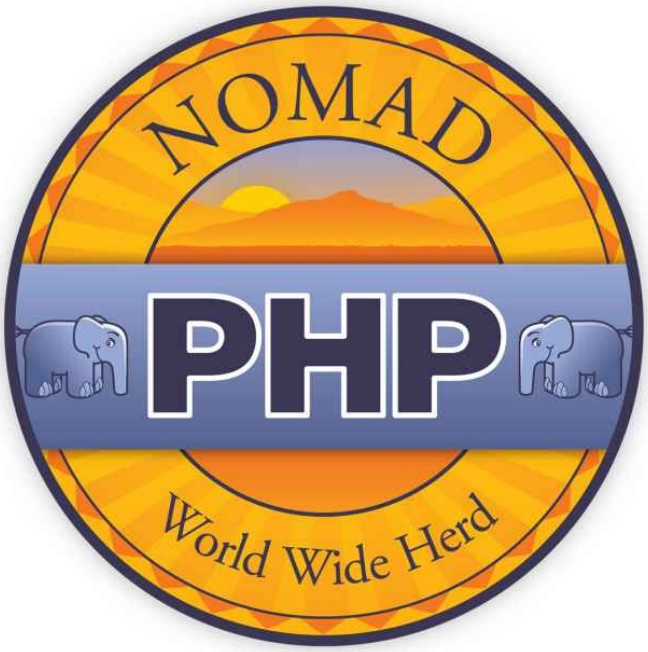
You can see that we have filtered the output so that we are only seeing the Other flags, and that both of them are set.

Summary

This has been a very quick primer into a complex subject. A lot of this is academic because unless you are working with “packed bits” like above, bit math isn’t done much in web programming. There are other programing specialties that make heavy use of this. If you decide to move into those areas, get a better explanation.

- To set bit flags we `OR` (`|`) values together.
- To check a bit’s value, we mask it with the `AND` (`&`) operator.

NomadPHP



Three Truths of Software Development

1. You have to keep learning
2. Your most important job is managing your career
3. You can fit learning into your busy schedule

You don't have to give up the things that are important to make time to learn. Each month we bring you new opportunities to learn and grow as a PHP developer.

Take a step forward in your career. Sign up for our free mailing list today and become a part of the World Wide Herd.

nomadphp.com